PRIMAcquisition

PRIMAcquisition (PRIM) is a Python-based application for synchronized acquisition of pressure data (from an Arduino-controlled pressure transducer) and live camera imaging. Designed for vascular physiology experiments, PRIMAcquisition displays live pressure traces, previews a high-speed camera feed, and saves synchronized recordings (as a CSV and TIFF stack) for offline analysis.

🚀 Quick Start – Run an Experiment

1️⃣ **Connect PRIM Device** – Select the Arduino COM port and click **Connect PRIM Device**

2️⃣ **Set Up Camera** – Choose camera & resolution → click **Start Camera**

3️⃣ **Adjust Exposure/Gain** – Use controls to fine-tune camera settings

4️⃣ **Zero PRIM** – Make sure pressure is at zero

5️⃣ **Start Recording** – Click **Start Recording** to begin acquisition

6️⃣ **Stop Recording** – Click **Stop Recording** when finished

7️⃣ **Playback & Export** – Click **Playback** to review, export a frame (PNG) or full TIFF with overlay

Features

🎥 Real-Time Pressure + Video Recording
- **Arduino is the master clock**:
  - Generates precise trigger pulses (`CamTrig`) to the camera for each frame.
  - Immediately sends synchronized serial data: `frame_index, elapsed_time_s, pressure_value`.
- **App listens for first Arduino tick**, then begins writing data.
- **Perfect sync** is maintained:
  - Each TIFF frame corresponds to exactly one Arduino trigger.
  - Each pressure value corresponds to exactly one recorded frame.
  - The matching pressure value is stored in each TIFF frame's metadata.

📈 Live Pressure Plotting
- Receives pressure readings from Arduino (115200 baud).
- Displays live pressure trace in real time.
- Shows frame index, elapsed time, and pressure in the top panel.

🎥 High-Speed Camera Preview & Control
- Integrates with The Imaging Source DMK cameras via IC Imaging Control 4 (IC4).
- Lists connected USB3 Vision cameras with supported resolutions.
- Displays a full-resolution preview using OpenGL viewfinder.
- Camera settings (exposure, gain, brightness) adjustable via sliders.

💾 Synchronized Recording Output
- Folder structure:
```
PRIM_ROOT/YYYY-MM-DD/FillN/
├ recording_*.csv     ← Pressure + frame index
└ recording_*.tif     ← Grayscale video, one frame per Arduino trigger
```
- Files are automatically named and saved in time-stamped subfolders.
- Environment variable `PRIM_RESULTS_DIR` overrides default save location.
- After recording you can open **Playback** to view the TIFF with pressure overlay (toggleable), export an annotated copy, or save a single-frame snapshot.

🛠️ Under the Hood

PRIMAcquisition uses a **hardware-triggered acquisition model**:

Component	Role
Arduino	⏱️ Master clock; sends trigger pulses and serial data
Camera	🎥 Triggered via `CamTrig` pin from Arduino
App	🧠 Waits for first Arduino message, then saves video + CSV

Each cycle:
1. Arduino triggers a camera frame via digital HIGH → LOW on `CamTrig`.
2. Arduino immediately sends pressure data and timestamp via serial.
3. App receives both the frame and serial line, saves them together.

Result: **pixel-perfect and time-accurate alignment** of pressure + video frames.

- **SerialThread** – Reads Arduino serial data and emits `(frameIndex, timestamp, pressure)`.
- **SDKCameraThread** – Opens IC4 camera, configures settings, streams frames to the GUI.
- **RecordingManager** – Writes synchronized CSV and TIFF files in a

background thread.
- **PlaybackWindow** – Reloads recorded files and overlays pressure data on frames.

Recording starts when the first Arduino tick is received. Each pressure value is embedded in its corresponding TIFF frame metadata, ensuring **perfect synchronization**.

Installation

1. **Clone the Repository**
 ```bash
 git clone https://github.com/your-repo/PRIMAcquisition.git
 cd PRIMAcquisition/prim_app
```

2. **Create a Virtual Environment**

   ```bash
   python -m venv .venv
   .venv\Scripts\activate   # On Windows
   ```

3. **Install Dependencies**

   ```bash
   pip install -r requirements.txt
   ```

   Or manually:

   ```bash
   pip install pyqt5 imagingcontrol4 pyserial numpy tifffile
   ```

4. **Install IC4 SDK**

   * Required for DMK camera support.
   * Ensure GenTL Producer is installed and camera enumerates via IC4.

Running the App

1. **Start App**

   ```bash
   python prim_app.py
   ```

   ```

2. **Connect Arduino**

   * Select COM port (e.g., COM8), then click **Connect PRIM Device**.
   * Confirm status shows “Connected”.

3. **Configure Camera**

   * Select camera and resolution.
   * Click **Start Camera** to preview live feed.

4. **Start Recording**

   * Use menu: **Acquisition → Start Recording** or press **Ctrl+R**.
   * App waits for first Arduino `'T...'` tick to begin synchronized recording.
   * TIFF and CSV are saved in the correct folder.

5. **Stop Recording**

   * Use menu: **Acquisition → Stop Recording** or press **Ctrl+T**.
   * Files are finalized and closed cleanly.

---

## Arduino Firmware (PRIM_v3_02)

* Loops every `startup.timeDelay` milliseconds.
* Sends a `CamTrig` HIGH–LOW pulse to trigger one camera frame.
* Immediately sends formatted serial data:

  ```
  frame_index, elapsed_time_s, pressure_value
  ```
* Operates with precise timing via `micros()` and `millis()`.

---

## Troubleshooting

| Issue | Fix |
| --- | --- |
| Camera not listed | Verify IC4 SDK + GenTL Producer installed |
| Serial shows no data | Check Arduino COM port and baud rate |
| TIFF won’t open | Use ImageJ/Fiji or Python `tifffile` |
| Dropped frames | Use USB 3.0 and reduce resolution if needed |

---
  ```

License